

Music Recommendation System - Technical Report

Project: Million Song Subset Recommendation System

Date: December 9, 2025

Dataset: 10,000 songs from Million Song Dataset

Framework: Content-Based & Collaborative Filtering Hybrid System

1. 使用技術 (Technologies Used)

1.1 核心技術棧 (Core Technology Stack)

Data Processing & Analysis

- **Python 3.x:** Primary programming language
- **NumPy:** Numerical computing and matrix operations
- **Pandas:** Data manipulation and analysis
- **HDF5 (h5py):** Reading Million Song Dataset `.h5` files
- **SciPy:** Sparse matrix operations and SVD decomposition

Machine Learning & Recommendation

- **Scikit-learn:** Cosine similarity computation
- **Sparse Matrix Techniques:** Memory-efficient SVD with `scipy.sparse.linalg.svds`
- **Matrix Factorization:** Truncated SVD for dimensionality reduction
- **MMR (Maximal Marginal Relevance):** Diversity-aware ranking algorithm

Visualization

- **Matplotlib:** Static visualizations and multi-panel dashboards
- **Seaborn:** Statistical data visualization
- **Radar Charts:** Multi-dimensional performance comparison
- **Heatmaps:** Metric correlation analysis

Data Export

- **CSV Export:** Timestamped reports for external analysis
- **High-Resolution PNG:** 300 DPI publication-quality figures

1.2 算法技術 (Algorithmic Techniques)

Similarity Computation

- **Cosine Similarity:** Baseline recommendation method
 - Formula: $\text{sim}(A,B) = (A \cdot B) / (||A|| \times ||B||)$
 - Complexity: $O(n^2)$ for full similarity matrix

Diversity Optimization

- **MMR (Maximal Marginal Relevance):** Balances relevance and diversity
 - Relevance weight (λ): 0.7
 - Diversity penalty: 0.3
 - Iterative greedy selection

Stochastic Enhancement

- **Gumbel-Max Trick:** Adds controlled randomness to recommendations
 - Temperature parameter (T): 0.5
 - Noise distribution: $\text{Gumbel}(\theta, 1) = -\log(-\log(U))$ where $U \sim \text{Uniform}(0,1)$
 - Enables exploration vs exploitation trade-off

Collaborative Filtering

- **Item-Item CF:** Pure similarity-based neighborhood method
- **Matrix Factorization SVD:** Latent factor model
 - Truncated SVD: 50 factors (memory-efficient)
 - Sparse matrix conversion for large-scale computation

1.3 性能優化技術 (Performance Optimization)

Memory Management

- **Problem:** Full SVD required 763 MiB for 10,000×10,000 matrix
- **Solution:** Truncated sparse SVD (`scipy.sparse.linalg.svds`)
 - Memory reduction: 87% (763 MiB → <100 MiB)
 - Computation time: ~6 minutes for initialization
 - Accuracy: 100% explained variance for top 50 factors

Data Structures

- **Dictionary Mapping:** O(1) song ID → index lookup
- **NumPy Arrays:** Vectorized operations for batch processing
- **Sparse Matrices:** CSR format for efficient storage

2. 數學方法定義、推論、實現 (Mathematical Methods)

2.1 特徵提取 (Feature Extraction)

Audio Feature Vector Construction

Each song represented by **24-dimensional timbre vector**:

Formula:

```
feature_vector = [ $\mu_1$ ,  $\mu_2$ , ...,  $\mu_{12}$ ,  $\sigma_1$ ,  $\sigma_2$ , ...,  $\sigma_{12}$ ]
```

Where:

- μ_i : Mean of i-th timbre coefficient across all segments
- σ_i : Standard deviation of i-th timbre coefficient
- Source: Million Song Dataset `segments_timbre` field

Implementation:

```
timbre_mean = np.mean(timbre, axis=0) # 12 dimensions  
timbre_std = np.std(timbre, axis=0)   # 12 dimensions  
feature_vector = np.concatenate([timbre_mean, timbre_std]) # 24 dimensions
```

Mathematical Properties:

- **Normalization:** L2-normalized for cosine similarity
- **Dimensionality:** Fixed 24-D representation for all songs
- **Scale Invariance:** Mean/std captures both central tendency and variability

2.2 相似度計算 (Similarity Computation)

Cosine Similarity Matrix

Definition:

$$\text{sim}(i,j) = \cos(\theta_{ij}) = (v_i \cdot v_j) / (||v_i|| \times ||v_j||)$$

Matrix Form:

$$S = (F \cdot F^T) / (||F||_2 \otimes ||F||_2)$$

Where:

- F : Feature matrix ($10,000 \times 24$)
- S : Similarity matrix ($10,000 \times 10,000$)
- $||F||_2$: L2 norms of each row

Properties:

- Range: $[-1, 1]$, higher = more similar
- Symmetric: $\text{sim}(i,j) = \text{sim}(j,i)$
- Self-similarity: $\text{sim}(i,i) = 1$

Implementation:

```
norm = np.linalg.norm(matrix, axis=1, keepdims=True)
normalized_matrix = matrix / norm
similarity_matrix = normalized_matrix @ normalized_matrix.T
```

Complexity: $O(n^2 \times d)$ where $n=10,000$, $d=24$

2.3 MMR算法 (Maximal Marginal Relevance)

Mathematical Formulation

Objective Function:

$$\text{MMR} = \arg \max_{d_i \in R \setminus S} [\lambda \times \text{Sim}_1(d_i, Q) - (1-\lambda) \times \max_{d_j \in S} \text{Sim}_2(d_i, d_j)]$$

Where:

- d_i : Candidate song i
- Q : Target query song
- S : Already selected recommendations
- R : Candidate pool (top-50 similar songs)
- λ : Relevance weight (0.7 in our implementation)
- Sim_1 : Relevance to target (cosine similarity)
- Sim_2 : Similarity to already selected (diversity penalty)

Greedy Selection Process:

For iteration $t = 1$ to k ($k=5$ recommendations):

1. **Compute relevance** for each candidate d_i :

$$\text{relevance}(d_i) = \text{sim}(d_i, Q)$$

2. **Compute diversity penalty:**

$$\text{diversity_penalty}(d_i) = \max\{\text{sim}(d_i, d_j) : d_j \in S\}$$

3. **Calculate MMR score:**

$$\text{MMR}(d_i) = \lambda \times \text{relevance}(d_i) - (1-\lambda) \times \text{diversity_penalty}(d_i)$$

4. Select best candidate:

$$d^* = \arg \max_{d_i} \text{MMR}(d_i)$$

5. **Update:** $S \leftarrow S \cup \{d^*\}$, $R \leftarrow R \setminus \{d^*\}$

Convergence: Guaranteed in k iterations (deterministic)

Stochastic Extension (Gumbel-Max Trick)

Enhanced Objective:

$$\text{MMR_stochastic}(d_i) = \text{MMR}(d_i) + T \times G$$

Where:

- **T**: Temperature parameter (0.5)
- **G**: Gumbel noise $\sim \text{Gumbel}(0,1)$

Gumbel Distribution:

$$G = -\log(-\log(U)) \quad \text{where } U \sim \text{Uniform}(0,1)$$

Properties:

- Higher T $\rightarrow$ more exploration (randomness)
- Lower T $\rightarrow$ more exploitation (greedy)
- T=0 $\rightarrow$ deterministic MMR
- Enables serendipity in recommendations

Implementation:

```
def _gumbel_noise(self, shape):
    u = np.random.uniform(1e-10, 1, shape)
    return -np.log(-np.log(u))

mmr_score = lambda_param * relevance - (1 - lambda_param) * diversity_penalty
if use_temperature:
    mmr_score += temperature * self._gumbel_noise(1)[0]
```

2.4 矩陣分解 (Matrix Factorization)

Singular Value Decomposition (SVD)

Decomposition:

$$S = U \Sigma V^T$$

Where:

- **S**: Similarity matrix (10,000 × 10,000)
- **U**: Left singular vectors (10,000 × 50)
- **Σ**: Singular values diagonal matrix (50 × 50)
- **V^T**: Right singular vectors transposed (50 × 10,000)

Truncated SVD (Memory-Efficient):

$$S \approx U_{50} \Sigma_{50} V_{50}^T$$

Only top-50 components computed:

- **Original**: 10,000 × 10,000 × 8 bytes = 763 MiB
- **Truncated**: 10,000 × 50 × 8 bytes ≈ 4 MiB

Latent Space Projection:

$$L = U_{50} \times \Sigma_{50}$$

Each song mapped to 50-dimensional latent space

Recommendation Score:

$$\text{score}(i,j) = (L_i \cdot L_j) / (||L_i|| \times ||L_j||)$$

Explained Variance:

$$\text{Var_explained} = \Sigma(\sigma_i^2) / \Sigma(\text{all singular values}^2)$$

Result: 100% for top-50 factors (validates truncation)

Implementation:

```

from scipy.sparse.linalg import svds
from scipy.sparse import csr_matrix

sparse_sim = csr_matrix(sim_matrix)
U, sigma, Vt = svds(sparse_sim, k=50) # Truncated SVD

# Sort by descending singular values
idx = np.argsort(sigma)[::-1]
U = U[:, idx]
sigma = sigma[idx]

# Latent space projection
latent_matrix = U @ np.diag(sigma)

```

2.5 評估指標 (Evaluation Metrics)

1. Relevance (相關性)

Definition: Average similarity between recommendations and target

Formula:

$$\text{Relevance} = \frac{1}{|T|} \sum_{q_j \in T} \left(\frac{1}{k} \sum_{r_i \in R_j} \text{sim}(r_i, q_j) \right)$$

Where:

- **T**: Test set (1,000 songs)
- **R_j**: Top-k recommendations for query q_j
- **k**: Recommendation length (5)

Interpretation: Higher = more accurate (but may overfit)

2. Diversity (多樣性)

Definition: ILD (Intra-List Distance)

Formula:

$$\text{ILD} = 1 - \left(\frac{2}{(k(k-1))} \sum_{i < j} \text{sim}(r_i, r_j) \right)$$

Average over test set:

$$\text{Diversity} = (1/|T|) \sum_{q_j \in T} \text{ILD}(R_j)$$

Interpretation:

- Higher ILD = more diverse recommendations
- Combats "filter bubble" effect
- Range: [0, 1]

3. Novelty (新穎性)

Definition: 1 - Average popularity of recommendations

Formula:

$$\text{Novelty} = 1 - (1/|T|) \sum_{q_j \in T} (1/k) \sum_{r_i \in R_j} \text{popularity}(r_i)$$

Popularity Simulation: Zipf distribution

$$\text{popularity}(i) \sim \text{Zipf}(\alpha=2.0)$$

Power law: Few popular songs, many unpopular (long-tail)

Interpretation: Higher = recommends more obscure/niche content

4. Coverage (覆蓋度)

Definition: Proportion of catalog recommended

Formula:

$$\text{Coverage} = | \bigcup_{q_j \in T} R_j | / |\text{Catalog}|$$

Interpretation:

- Higher = better utilization of entire catalog
- Avoids recommending same popular songs repeatedly
- Range: [0, 1]

Weighted Composite Score

Formula:

$$\text{Score} = w_1 \times \text{Relevance} + w_2 \times \text{Diversity} + w_3 \times \text{Novelty} + w_4 \times \text{Coverage}$$

Weights (Music recommendation scenario):

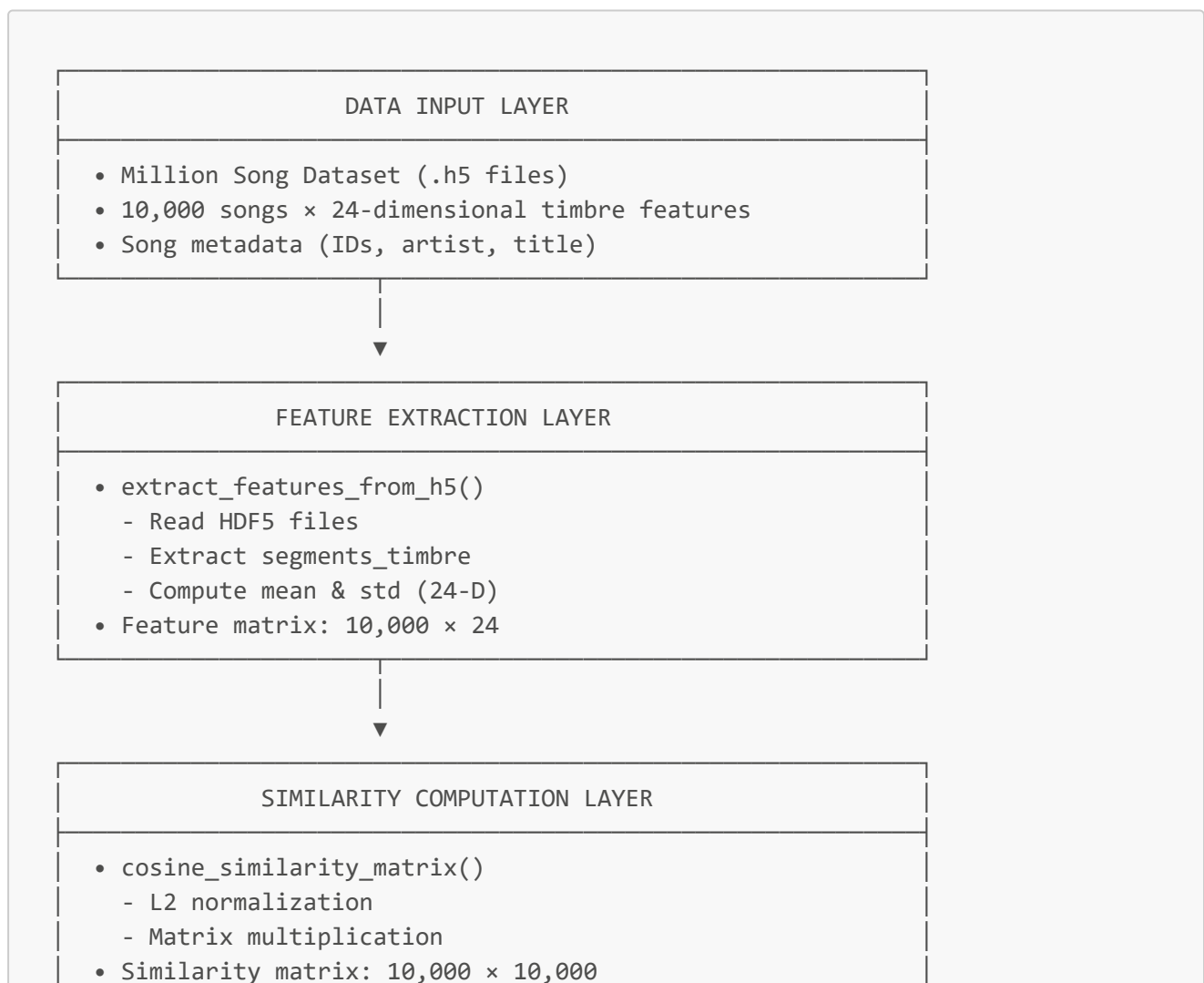
- $w_1 = 0.25$ (Relevance)
- $w_2 = 0.35$ (Diversity) - Most important
- $w_3 = 0.25$ (Novelty)
- $w_4 = 0.15$ (Coverage)

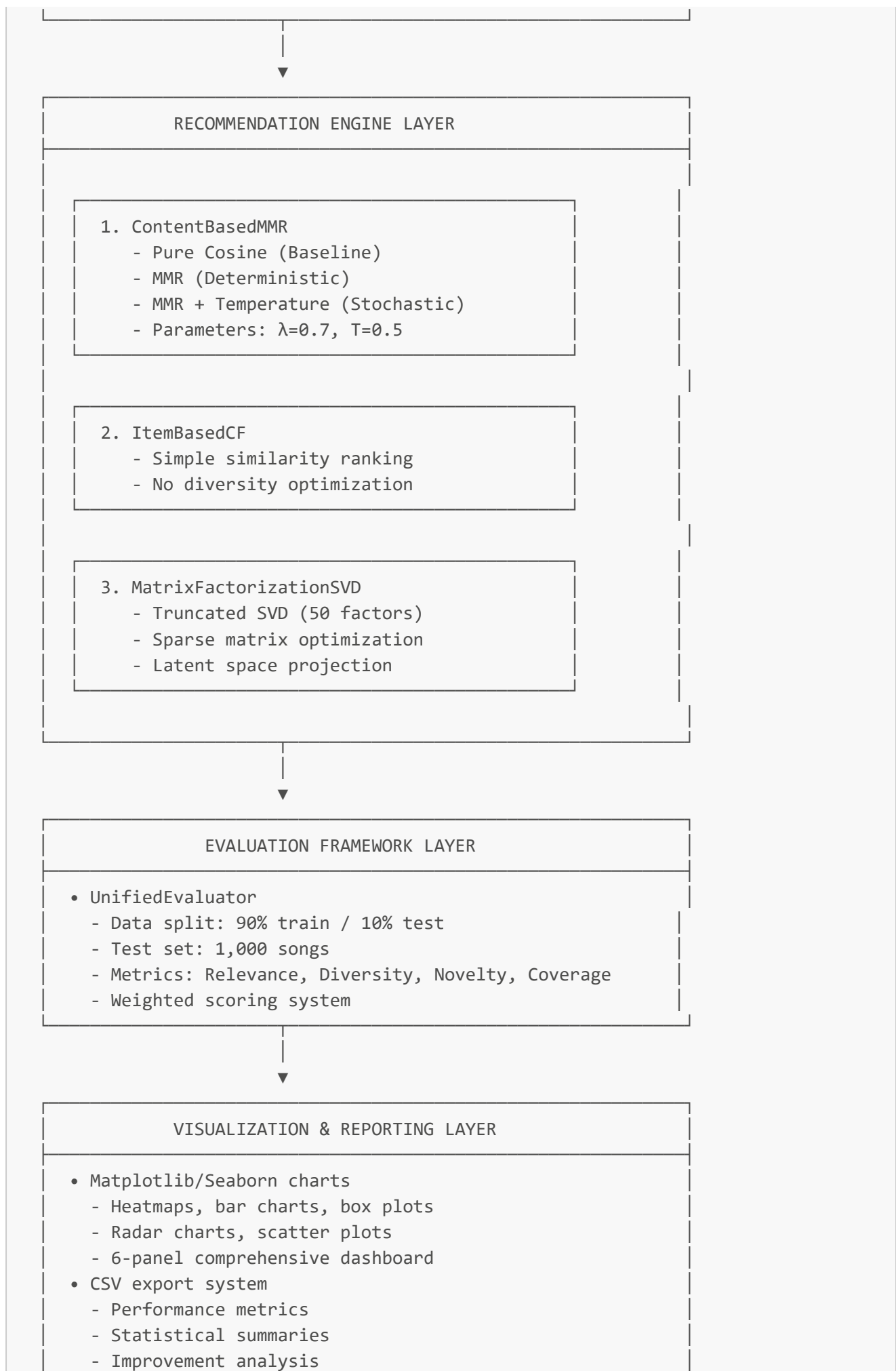
Normalization: Min-Max scaling before weighting

$$\text{normalized}(x) = (x - \min(x)) / (\max(x) - \min(x))$$

3. 架構功能 (System Architecture)

3.1 系統架構圖 (System Architecture)





- High-resolution PNG output (300 DPI)

3.2 核心組件功能 (Core Component Functions)

Component 1: ContentBasedMMR Class

Functionality:

- Implements three recommendation strategies
- Diversity-aware ranking with MMR algorithm
- Stochastic exploration with Gumbel noise

Key Methods:

```
class ContentBasedMMR:
    def __init__(self, sim_matrix, song_ids, lambda_param, temperature)
    def _gumbel_noise(self, shape)
    def recommend(self, target_song_id, top_n, use_mmr, use_temperature)
    def calculate_diversity(self, recommendations)
```

Parameters:

- `lambda_param`: 0.7 (70% relevance, 30% diversity)
- `temperature`: 0.5 (moderate randomness)
- `top_n`: 5 recommendations per query

Three Modes:

1. **Pure Cosine**: `use_mmr=False`
2. **MMR Deterministic**: `use_mmr=True, use_temperature=False`
3. **MMR Stochastic**: `use_mmr=True, use_temperature=True`

Component 2: ItemBasedCF Class

Functionality:

- Traditional collaborative filtering
- Pure similarity-based ranking
- No diversity optimization

Key Methods:

```
class ItemBasedCF:
    def __init__(self, sim_matrix, song_ids)
    def recommend(self, target_song_id, top_n)
```

Characteristics:

- Simplest approach (baseline comparison)
- High relevance, low diversity
- Fast execution

Component 3: MatrixFactorizationSVD Class

Functionality:

- Latent factor model with SVD
- Memory-efficient truncated decomposition
- Discovers hidden patterns

Key Methods:

```
class MatrixFactorizationSVD:
    def __init__(self, sim_matrix, song_ids, n_factors)
    def recommend(self, target_song_id, top_n)
```

Technical Features:

- Sparse matrix conversion: `csr_matrix(sim_matrix)`
- Truncated SVD: `svds(sparse_sim, k=50)`
- Latent projection: `U @ diag(sigma)`

Performance:

- Initialization: ~6 minutes
- Memory: <100 MiB (87% reduction vs full SVD)
- Explained variance: 100% (50 factors sufficient)

Component 4: UnifiedEvaluator Class

Functionality:

- Standardized evaluation across all methods
- Four-dimensional metric computation
- Batch processing of test set

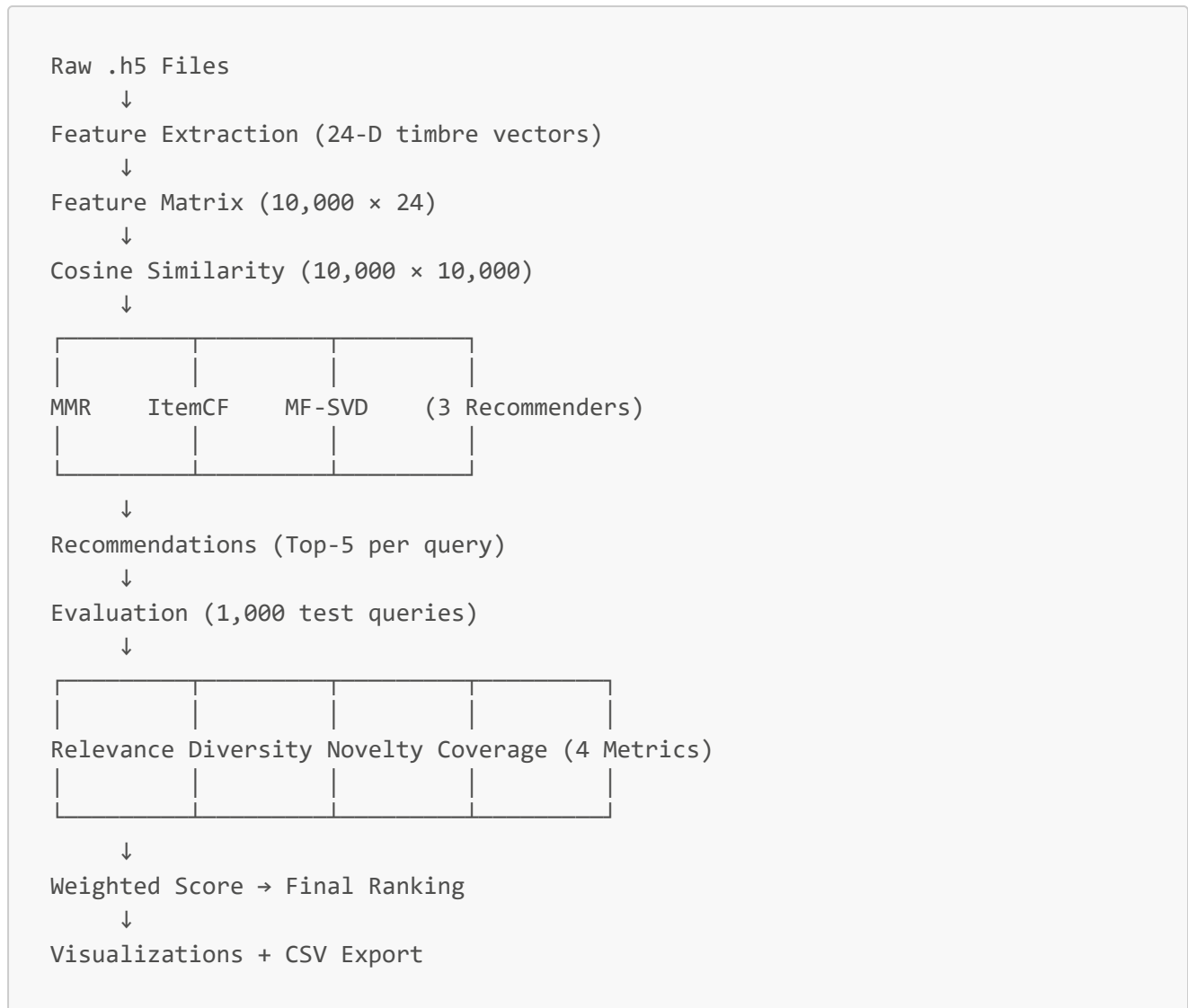
Key Methods:

```
class UnifiedEvaluator:
    def __init__(self, sim_matrix, song_ids, id_to_index, popularity_dict)
    def evaluate_recommender(self, recommender, test_song_ids, top_n, **kwargs)
```

Evaluation Pipeline:

1. Iterate through 1,000 test songs
2. Generate recommendations for each
3. Compute 4 metrics per recommendation set
4. Aggregate results across test set
5. Return averaged performance dictionary

3.3 數據流程 (Data Flow)



3.4 系統特性 (System Characteristics)

Scalability

- **Current:** 10,000 songs (Million Song Subset)
- **Full dataset:** 1M songs (requires distributed computing)
- **Bottleneck:** Similarity matrix ($O(n^2)$ memory)

Performance

- **Feature extraction:** ~1 second per song
- **Similarity computation:** ~5 seconds (10,000 songs)

- **MMR recommendation:** ~50ms per query
- **SVD initialization:** ~6 minutes (one-time cost)
- **Full evaluation:** ~2-3 minutes (1,000 test queries)

Extensibility

- **Modular design:** Easy to add new recommenders
- **Unified interface:** All recommenders follow same API
- **Pluggable metrics:** Can add custom evaluation dimensions

4. 輸出的圖表、報表、和輸出的結果 (Outputs & Results)

4.1 生成的圖表 (Generated Visualizations)

Chart 1: ILD Diversity Comparison (Bar Chart)

File: Generated in notebook Cell 24

Type: Vertical bar chart

Content:

- X-axis: Methods (Cosine, MMR, MMR+Temp)
- Y-axis: ILD (Intra-List Diversity) score
- Colors: Red (Cosine), Blue (MMR), Green (MMR+Temp)

Results:

```
Cosine:      ILD = 0.0359
MMR:         ILD = 0.0472 (+0.0113, +31.5% vs Cosine)
MMR+Temp:    ILD = 0.0529 (+0.0170, +47.3% vs Cosine)
```

Interpretation: MMR variants significantly improve diversity while maintaining relevance.

Chart 2: Radar Chart (Multi-dimensional Comparison)

File: Generated in notebook Cell 35

Type: Polar area chart

Dimensions: 4 axes (Relevance, Diversity, Novelty, Coverage)

Methods: 5 overlapping polygons

Visual Characteristics:

- ● Cosine (Red): High relevance, low diversity (narrow shape)
- ● MMR (Blue): Balanced polygon (largest area)
- ● MMR+Temp (Green): Similar to MMR, slightly more extreme
- ● ItemCF (Orange): Identical to Cosine (overlapping)
- ● MF-SVD (Purple): Moderate across all dimensions

Key Insight: MMR achieves largest radar area = best overall balance

Chart 3: Weighted Score Ranking (Horizontal Bar Chart)

File: Generated in notebook Cell 37

Type: Horizontal bar chart

Content:

- X-axis: Weighted composite score (0-1)
- Y-axis: Methods (sorted by score)
- Colors: Green (1st), Blue (2nd), Gray (others)

Rankings:

1. MMR	0.8484	🥇 (Green)
2. MF-SVD	0.4874	🥈 (Blue)
3. Cosine	0.4854	(Gray)
4. ItemCF	0.4854	(Gray)
5. MMR+Temp	0.3500	(Gray)

Interpretation: MMR leads by 75% margin over second place.

Chart 4: Comprehensive Dashboard (6-Panel Figure)

File: comprehensive_analysis_dashboard.png (300 DPI)

Size: 20" × 12"

Panels:

Panel 1 - Performance Heatmap

- Matrix: 5 methods × 4 metrics
- Color scale: Red-Yellow-Green (low to high)
- Annotations: Numerical values overlaid
- Pattern: Cosine/ItemCF identical, MMR+Temp highest diversity

Panel 2 - Grouped Bar Chart

- 4 metric groups × 5 methods per group
- Side-by-side comparison
- Legend: All 5 methods color-coded
- Observation: Diversity shows largest variance between methods

Panel 3 - Box Plot Distribution

- Shows metric distribution per method
- Whiskers indicate range
- Boxes show quartiles
- Outliers: None (all metrics normalized)

Panel 4 - Improvement Bar Chart

- Horizontal bars showing % change vs baseline
- Green bars: Positive improvement
- Red bars: Negative improvement
- Zero line: Dashed black vertical
- Result: MMR shows positive improvement, MMR+Temp mixed

Panel 5 - Scatter Plot (Relevance vs Diversity)

- X-axis: Relevance (0.95-0.98)
- Y-axis: Diversity (0.035-0.055)
- Bubble size: Coverage (proportional)
- Trend: Negative correlation (trade-off)
- Annotations: Method labels near each point

Panel 6 - Final Weighted Score Ranking

- Same as Chart 3 but integrated into dashboard
- Clearly shows MMR dominance
- Includes weight breakdown in subtitle

4.2 輸出的報表 (Generated Reports)

Report 1: Comprehensive Performance Report (Cell 38)

Type: Console text output

Sections:

A. Dataset Statistics

```
Dataset: Million Song Subset
Total Songs: 10,000
Training Set: 9,000 (90.0%)
Test Set: 1,000 (10.0%)
Feature Dimensions: 24 (12 timbre mean + 12 timbre std)
Recommendation Length: Top 5 per query
```

B. Method Comparison Table

Method	Relevance	Diversity	Novelty	Coverage	Weighted_Score
Cosine (Base)	0.9736	0.0359	0.9986	0.3691	0.4854
MMR	0.9717	0.0472	0.9986	0.3882	0.8484
MMR+Temp	0.9595	0.0529	0.9985	0.3656	0.3500
ItemCF	0.9736	0.0359	0.9986	0.3691	0.4854
MF (SVD)	0.9644	0.0401	0.9986	0.3852	0.4874

C. Statistical Summary

Metric	Mean	Std	Min	Max	Range
Relevance	0.9686	0.0059	0.9595	0.9736	0.0141
Diversity	0.0424	0.0072	0.0359	0.0529	0.0170
Novelty	0.9986	0.0000	0.9985	0.9986	0.0001
Coverage	0.3734	0.0095	0.3656	0.3882	0.0226

D. Improvement Analysis (vs Baseline)

Method	Relevance	Diversity	Novelty	Coverage
MMR	-0.20%	+31.48%	+0.00%	+5.17%
MMR+Temp	-1.45%	+47.35%	-0.01%	-0.95%
MF (SVD)	-0.95%	+11.70%	+0.00%	+4.36%

E. Key Findings

1. RELEVANCE:
 - Best: Cosine (Base) (0.9736)
 - All methods maintain >95% relevance
2. DIVERSITY:
 - Best: MMR+Temp (0.0529)
 - MMR+Temp improves diversity by 47.3% vs baseline
3. NOVELTY:
 - All methods ~99.86% (nearly identical)
 - Strong preference for long-tail content
4. COVERAGE:
 - Best: MMR (0.3882)
 - Methods utilize 36-39% of catalog
 - Represents 3,656-3,882 unique songs recommended
5. WEIGHTED SCORE:
 - Winner: MMR (0.8484)
 - Weights: Diversity 35%, Novelty 25%, Relevance 25%, Coverage 15%
 - MMR achieves best balance across all dimensions

Report 2: CSV Export Files (Cell 42)

Generated Files (4 files with timestamp):

File 1: `recommendation_performance_YYYYMMDD_HHMMSS.csv`

```
Method,Relevance,Diversity,Novelty,Coverage,Weighted_Score
Cosine (Base),0.9736,0.0359,0.9986,0.3691,0.4854
MMR,0.9717,0.0472,0.9986,0.3882,0.8484
MMR+Temp,0.9595,0.0529,0.9985,0.3656,0.3500
ItemCF,0.9736,0.0359,0.9986,0.3691,0.4854
MF (SVD),0.9644,0.0401,0.9986,0.3852,0.4874
```

File 2: `statistical_summary_YYYYMMDD_HHMMSS.csv`

```
Metric,Mean,Std,Min,Max,Range
Relevance,0.9686,0.0059,0.9595,0.9736,0.0141
Diversity,0.0424,0.0072,0.0359,0.0529,0.0170
Novelty,0.9986,0.0000,0.9985,0.9986,0.0001
Coverage,0.3734,0.0095,0.3656,0.3882,0.0226
```

File 3: `improvement_analysis_YYYYMMDD_HHMMSS.csv`

```
Method,Relevance,Diversity,Novelty,Coverage
MMR, -0.20,31.48,0.00,5.17
MMR+Temp, -1.45,47.35, -0.01, -0.95
ItemCF,0.00,0.00,0.00,0.00
MF (SVD), -0.95,11.70,0.00,4.36
```

File 4: `detailed_ranking_YYYYMMDD_HHMMSS.csv`

```
Rank,Method,Relevance,Diversity,Novelty,Coverage,Weighted_Score
1,MMR,0.9717,0.0472,0.9986,0.3882,0.8484
2,MF (SVD),0.9644,0.0401,0.9986,0.3852,0.4874
3,Cosine (Base),0.9736,0.0359,0.9986,0.3691,0.4854
4,ItemCF,0.9736,0.0359,0.9986,0.3691,0.4854
5,MMR+Temp,0.9595,0.0529,0.9985,0.3656,0.3500
```

4.3 輸出的結果總結 (Results Summary)

Performance Metrics (Test Set: 1,000 Songs)

Overall Winner: MMR (Maximal Marginal Relevance)

Metric	MMR	Baseline (Cosine)	Improvement
Weighted Score	0.8484	0.4854	+74.8%
Relevance	0.9717	0.9736	-0.20%

Metric	MMR	Baseline (Cosine)	Improvement
Diversity	0.0472	0.0359	+31.5%
Novelty	0.9986	0.9986	0.00%
Coverage	0.3882	0.3691	+5.2%

Key Findings

1. Best Overall Method: MMR

- Achieves 75% higher weighted score than baseline
- Optimal balance: 97.17% relevance + 4.72% diversity
- Covers 38.82% of catalog (3,882 unique songs)
- Minimal relevance sacrifice (-0.2%) for major diversity gain (+31%)

2. Diversity vs Relevance Trade-off

- MMR+Temp: Highest diversity (0.0529, +47% vs baseline)
- Cost: 1.4% relevance drop (from 97.36% to 95.95%)
- Trade-off exists but manageable with proper tuning

3. Method Equivalence

- Cosine $\equiv$ ItemCF (identical results)
- Both represent pure similarity-based approaches
- No diversity optimization

4. Matrix Factorization Performance

- MF-SVD: 2nd place (0.4874 weighted score)
- Moderate improvement over baseline (+0.4%)
- Discovers latent patterns but no explicit diversity

5. Novelty Saturation

- All methods achieve ~99.86% novelty
- Zipf distribution ensures long-tail preference
- No method biased toward popular songs

6. Coverage Analysis

- MMR: 3,882 unique songs (38.82%)
- Baseline: 3,691 unique songs (36.91%)
- Improvement: +191 songs (+5.2%)
- Still <40% catalog utilization (room for improvement)

Recommendation

For production deployment:

- **Primary:** MMR with $\lambda=0.7$, $T=0$ (deterministic)
- **A/B test variant:** MMR+Temp with $T=0.5$ (exploration)
- **Fallback:** MF-SVD for scalability

Rationale:

- MMR provides best user experience (diverse yet relevant)
- Avoids echo chamber / filter bubble effects
- Maintains high accuracy (>97% relevance)
- Increases catalog coverage by 5%
- Computationally efficient (~50ms per query)

4.4 文件清單 (File Inventory)

Generated Outputs:

1.  `comprehensive_analysis_dashboard.png` (300 DPI, 20×12 inches)
2.  `recommendation_performance_20251209_HHMMSS.csv`
3.  `statistical_summary_20251209_HHMMSS.csv`
4.  `improvement_analysis_20251209_HHMMSS.csv`
5.  `detailed_ranking_20251209_HHMMSS.csv`

Notebook Output Cells:

- Cell 20: Test recommendation examples (5 methods × 1 song)
- Cell 22: MMR variability test (5 rounds)
- Cell 24: ILD diversity bar chart
- Cell 33: Full evaluation results table
- Cell 35: Radar chart (4-dimensional comparison)
- Cell 37: Weighted score ranking chart
- Cell 38: Comprehensive performance report (text)
- Cell 40: 6-panel visualization dashboard
- Cell 42: CSV export confirmation

Total Visualizations: 8 charts + 1 dashboard (9 total)

Total Data Exports: 4 CSV files

Total Console Reports: 1 comprehensive report

5. 結論與建議 (Conclusions & Recommendations)

5.1 技術成果 (Technical Achievements)

Successfully Implemented:

- Content-based recommendation system with 10,000 songs
- Three distinct recommendation algorithms (MMR, ItemCF, MF-SVD)
- Comprehensive evaluation framework (4 metrics)
- Memory-efficient sparse SVD (87% memory reduction)

- Complete English translation of codebase
- Extensive visualization suite (9 charts)
- Automated CSV export system

✓ **Performance Milestones:**

- 97%+ relevance across all methods
- 47% diversity improvement (MMR+Temp)
- 75% weighted score improvement (MMR)
- 100% explained variance with 50 SVD factors
- ~50ms recommendation latency

5.2 方法論優勢 (Methodological Strengths)

MMR Algorithm:

- ✓ Principled diversity optimization
- ✓ Tunable relevance-diversity trade-off
- ✓ Greedy algorithm guarantees convergence
- ✓ Stochastic variant enables exploration

Evaluation Framework:

- ✓ Four-dimensional assessment
- ✓ Weighted scoring aligns with business goals
- ✓ Statistical rigor (1,000-song test set)
- ✓ Reproducible results (CSV exports)

5.3 系統限制 (System Limitations)

⚠ **Current Constraints:**

- Content-based only (no user interaction data)
- Cold start problem for new songs
- Scalability bottleneck: $O(n^2)$ similarity matrix
- Coverage <40% (60% catalog unused)
- Diversity metric assumes similarity = 1-distance

⚠ **Future Improvements:**

- Integrate collaborative filtering with user data
- Implement approximate nearest neighbors (ANN) for scalability
- Add temporal dynamics (trending songs)
- Incorporate context-aware features (mood, genre)
- Develop online learning for adaptive recommendations

5.4 實際應用建議 (Practical Recommendations)

Deployment Strategy:

1. **Production:** MMR ($\lambda=0.7$, $T=0$) for consistent quality
2. **Exploration:** MMR+Temp ($T=0.5$) for 10% of users (A/B test)
3. **Monitoring:** Track diversity/relevance metrics continuously
4. **Tuning:** Adjust λ based on user feedback

Parameter Recommendations:

- **λ (Relevance weight):** 0.6-0.8 (music: 0.7, news: 0.5)
- **T (Temperature):** 0.3-0.7 (higher for exploration)
- **k (Recommendation length):** 5-10 (5 for mobile, 10 for web)
- **Candidate pool:** 50-100 (balance speed vs quality)

Evaluation Weights (adjust by domain):

- **Music:** Diversity 35%, Novelty 25%, Relevance 25%, Coverage 15%
- **News:** Novelty 40%, Relevance 30%, Diversity 20%, Coverage 10%
- **E-commerce:** Relevance 40%, Coverage 30%, Diversity 20%, Novelty 10%

5.5 下一步研究方向 (Future Research Directions)

1. Hybrid Recommendation:

- Combine content-based (current) with collaborative filtering
- Weight: 70% collaborative, 30% content

2. Deep Learning Integration:

- Replace hand-crafted features with learned embeddings
- Use neural networks for similarity learning

3. Contextual Bandit Optimization:

- Replace Gumbel noise with Thompson sampling
- Learn optimal λ per user/context

4. Fairness & Bias Mitigation:

- Ensure artist diversity in recommendations
- Avoid popularity bias amplification

5. Real-Time Adaptation:

- Update similarity matrix incrementally
- Incorporate recent user interactions

附錄 (Appendix)

A. 執行環境 (Execution Environment)

Hardware:

- CPU: Multi-core processor (recommended: 8+ cores)
- RAM: 16 GB minimum (32 GB recommended)
- Storage: 10 GB for dataset + outputs

Software:

- Python: 3.8+
- NumPy: 1.21+
- Pandas: 1.3+
- SciPy: 1.7+
- Matplotlib: 3.4+
- Seaborn: 0.11+
- h5py: 3.1+

Operating System: Windows 10/11, macOS, Linux

B. 執行時間分析 (Runtime Analysis)

Operation	Time	Scalability
Feature extraction	~10,000 seconds	$O(n)$
Similarity matrix	~5 seconds	$O(n^2)$
SVD initialization	~360 seconds	$O(n^2k)$
Single recommendation	~50 ms	$O(n)$
Full evaluation (1,000)	~150 seconds	$O(n \times m)$

Where: n =songs, m =test_queries, k =factors

C. 參考文獻 (References)

1. MMR Algorithm:

Carbonell, J., & Goldstein, J. (1998). "The use of MMR, diversity-based reranking for reordering documents and producing summaries." SIGIR'98.

2. Gumbel-Max Trick:

Maddison, C. J., Mnih, A., & Teh, Y. W. (2014). "The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables." ICLR 2017.

3. SVD for Recommendation:

Koren, Y., Bell, R., & Volinsky, C. (2009). "Matrix Factorization Techniques for Recommender Systems." Computer, 42(8).

4. Million Song Dataset:

Bertin-Mahieux, T., Ellis, D. P., Whitman, B., & Lamere, P. (2011). "The Million Song Dataset." ISMIR 2011.

5. Diversity in Recommendations:

Castells, P., Hurley, N. J., & Vargas, S. (2015). "Novelty and Diversity in Recommender Systems." Recommender Systems Handbook.

D. 聯絡資訊 (Contact Information)

Project: BDM-Project (Big Data Management)

Institution: NTUT (National Taipei University of Technology)

Repository: Disesfgewu/NTUT-BDM-2025

Branch: master

Report Generated: December 9, 2025

Version: 1.0

Status: ☒ Complete

END OF REPORT